TUTORIAL DE GIT

Capítulo 7. Herramientas de Git

Juan Felipe Castañeda Rodríguez · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)
Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Más allá del flujo básico, Git incluye herramientas potentes para buscar en la historia, reescribirla, depurar errores y manejar casos especiales. Este capítulo presenta las más útiles.

7.1 Revisión por selección

Hay muchas formas de referirse a un commit:

```
git show 1c002dd
git show HEAD~2
git show HEAD^
git show main@{yesterday}
git reflog
```

- Un SHA-1 abreviado basta si es único (normalmente 7 caracteres).
- HEAD^ es el padre; HEAD~3 es el bisabuelo.
- git reflog guarda dónde ha estado HEAD en los últimos meses, útil para recuperar trabajo "perdido".

Rangos:

```
git log main..experimento
git log main...experimento --left-right
```

Dos puntos: commits en experimento que no están en main. Tres puntos: commits en cualquiera de las dos, pero no en ambas.

7.2 Preparación interactiva

```
git add -i
git add -p
```

Permiten preparar solo partes (hunks) de un archivo para crear commits más precisos.

7.3 Guardado rápido y limpieza

Stash. Guarda temporalmente cambios sin confirmar para cambiar de tarea:

```
git stash
git stash list
git stash apply
git stash pop
git stash drop
```

Clean. Elimina archivos sin rastrear (con cuidado):

```
git clean -n
git clean -f -d
```

-n muestra qué se borraría sin hacerlo.

7.4 Firmar el trabajo

Con GPG se pueden firmar etiquetas y commits para demostrar su autoría:

```
git tag -s v1.0 -m "Versión firmada"
git commit -S -m "Commit firmado"
git log --show-signature
```

7.5 Búsqueda

```
git grep -n funcion
git log -S nombre_variable --oneline
git log -L :mi_funcion:archivo.py
```

git grep busca en los archivos rastreados; git log -S encuentra en qué commit apareció o desapareció un texto.

7.6 Reescribir la historia

Solo debe hacerse con commits que aún no se han compartido.

```
git commit --amend
git rebase -i HEAD~3
```

El rebase interactivo abre una lista de commits donde se puede elegir: pick (mantener), reword (cambiar mensaje), edit (modificar), squash (combinar con el anterior), drop (eliminar) o reordenar líneas. Para cambios masivos existe git filter-branch, hoy reemplazado por la herramienta git filter-repo.

7.7 Reset desmitificado

Git maneja tres árboles: HEAD (último commit), el índice (área de preparación) y el directorio de trabajo. git reset mueve la rama actual y, según la opción, afecta a los demás:

- **--soft:** solo mueve la rama; los cambios quedan preparados.
- **--mixed** (por defecto): mueve la rama y actualiza el índice; los cambios quedan en el directorio de trabajo sin preparar.
- **--hard:** mueve la rama, el índice y el directorio de trabajo; los cambios se pierden.

```
git reset --soft HEAD~1
git reset HEAD~1
git reset --hard HEAD~1
```

7.8 Fusión avanzada

```
git merge --abort
git merge -Xours otra_rama
git checkout --conflict=diff3 archivo.txt
git revert -m 1 HEAD
```

git revert crea un commit nuevo que deshace otro, sin reescribir la historia; es la forma segura de deshacer algo ya publicado.

7.9 Rerere

"Reuse recorded resolution": Git recuerda cómo se resolvió un conflicto y lo aplica automáticamente si vuelve a aparecer.

```
git config --global rerere.enabled true
```

7.10 Depuración con Git

```
git blame -L 10,20 archivo.py
git bisect start
git bisect bad
git bisect good v1.0
git bisect reset
```

git blame muestra quién modificó cada línea por última vez. git bisect hace una búsqueda binaria en la historia para encontrar el commit que introdujo un error.

7.11 Submódulos

Permiten incluir otro repositorio dentro del propio manteniéndolos independientes:

```
git submodule add https://github.com/usuario/libreria.git
git clone --recurse-submodules https://github.com/usuario/proyecto.git
git submodule update --remote
```

7.12 Empaquetado y reemplazo

git bundle empaqueta un repositorio en un solo archivo para moverlo sin red; git replace permite sustituir un objeto por otro en la vista de la historia.

7.13 Almacenamiento de credenciales

```
git config --global credential.helper cache
git config --global credential.helper store
```

En Windows, el Git Credential Manager guarda las credenciales de GitHub de forma segura.

7.14 Resumen

Estas herramientas permiten navegar la historia con precisión, guardar trabajo temporal, reescribir commits locales, entender reset, resolver fusiones complejas y encontrar el origen de errores.

Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2